Министерство науки и высшего образования РФ
Федеральное государственное автономное
образовательное учреждение высшего образования
«СИБИРСКИЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ»

Институт космических и информационных технологий
институт

Программная инженерия
кафедра

ОТЧЕТ О ПРАКТИЧЕСКОЙ РАБОТЕ №5

по дисциплине «Серверное программирование»

Тема: сущности, Criteria API и репозитории

Преподаватель _____ _____ _____
подпись, дата инициалы, фамилия

Студент _____ _____ Mohamed Amini
номер группы, зачетной книжки | подпись, дата | инициалы, фамилия

Красноярск 2026

1 Цель и постановка задачи

Цель работы – реализовать модель данных в виде классов-сущностей, освоить построение запросов программным интерфейсом Criteria и перейти к декларативному доступу к данным через репозитории (аналог Spring Data JPA Repository). Работа продолжает проект практического задания №4 и использует его класс DAO.

Требования задания:

– создать сущности (Entity) для всех таблиц базы данных;

– создать DAO сущностей с методом findAll на основе Criteria API;

– переписать методы практического задания №4 на Criteria API;

– реализовать метод с динамическими фильтрами (условия добавляются только при наличии параметров);

– реализовать операции get, save, delete и демонстрацию связи «один ко многим»;

– создать репозитории основных сущностей;

– реализовать метод запроса по соглашению об именах (query method) и один метод с произвольным запросом (@Query).

Критерии приёмки: сущности для всех таблиц; запросы Criteria работают; динамический фильтр и демонстрация связи 1 : N; репозиторий подключён и используется; query method и @Query возвращают корректные данные. Отчёт содержит классы Entity, DAO и Repository, примеры вызовов и вывод в консоль.

2 Соответствие эталонного стека и использованного

Работа выполнена на Node.js и TypeScript с ORM Drizzle (альтернативный стек согласован с преподавателем). Соответствие элементов эталона приведено в таблице 1.

Таблица 1 – Соответствие Java и TypeScript

Эталон (Java)	**Реализация (TypeScript)**
@Entity, @Table, @Id, @Column	pgTable('имя', { столбцы }) в src/model.ts
@OneToMany, @ManyToOne	relations() в src/relations.ts: many и one

Эталон (Java)	Реализация (TypeScript)
Criteria, Restrictions	класс Criteria и объект Restrictions в src/Criteria.ts
DAO с методами findAll, get, save, delete	классы StationDao и DistrictDao
JpaRepository	обобщённый класс JpaRepository в src/JpaRepository.ts
query method findByNameStartingWithIgnoreCase	метод findByNameStartingWithIgnoreCase класса StationRepository
@Query с JOIN	метод findStationsByDistrictName класса StationRepository

3 Сущности (Entity)

Файл model.ts описывает все девять таблиц базы данных: district, station, bicycle, client, tariff, rental, payment, client_wallet и client_favorite_station. Для каждой таблицы заданы столбцы, их типы, первичный ключ, ограничения и внешние ключи; из описания выводятся типы записи для чтения и для вставки. Начало файла – сущности district и station:

```
import { sql } from 'drizzle-orm';
import { type AnyPgColumn, boolean, date, integer, numeric, pgTable, primaryKey, smallint, text, timestamp }
from 'drizzle-orm/pg-core';

export const district = pgTable('district', {
    id: integer().primaryKey().generatedByDefaultAsIdentity(),
    name: text().notNull().unique(),
    population: integer().notNull(),
    areaKm2: numeric('area_km2', { precision: 7, scale: 2 }).notNull(),
    isCentral: boolean('is_central').notNull().default(false),
});

export const station = pgTable('station', {
    id: integer().primaryKey().generatedByDefaultAsIdentity(),
    districtId: integer('district_id').notNull().references(() => district.id),
    name: text().notNull(),
    address: text().notNull(),
    capacity: smallint().notNull(),
    latitude: numeric({ precision: 9, scale: 6 }).notNull(),
    longitude: numeric({ precision: 9, scale: 6 }).notNull(),
    openedOn: date('opened_on').notNull(),
});
```

Связь «один ко многим» между районом и станциями описана в файле relations.ts с двух сторон: у района много станций, у станции один район.

```
import { relations } from 'drizzle-orm';
import { district, station } from './model.ts';

export const districtRelations = relations(district, ({ many }) => ({
    stations: many(station),
}));

export const stationRelations = relations(station, ({ one }) => ({
    district: one(district, { fields: [station.districtId], references: [district.id] }),
}));
```

4 DAO и Criteria API

Класс Criteria (файл Criteria.ts) повторяет идею Hibernate Criteria: условия и порядок сортировки накапливаются методами add и addOrder, затем метод list выполняет запрос. Объект Restrictions содержит условия eq, ge, le, between, like и ilike.

```
import { and, asc, between, desc, eq, gte, ilike, lte, like, type SQL } from 'drizzle-orm';
import type { PgTable } from 'drizzle-orm/pg-core';
import type { Session } from './DAO.ts';

export const Restrictions = { eq, ge: gte, le: lte, between, like, ilike };
export const Order = { asc, desc };

export class Criteria<T extends PgTable> {
    private session: Session;
    private table: T;
    private conditions: SQL[] = [];
    private orders: SQL[] = [];

    constructor(session: Session, table: T) {
        this.session = session;
        this.table = table;
    }

    add(condition: SQL): this {
        this.conditions.push(condition);
        return this;
    }

    addOrder(order: SQL): this {
        this.orders.push(order);
        return this;
    }

    async list(): Promise<Array<T['$inferSelect']>> {
        let query = this.session.select().from(this.table as PgTable).$dynamic();
        if (this.conditions.length > 0) query = query.where(and(...this.conditions));
        if (this.orders.length > 0) query = query.orderBy(...this.orders);
        return (await query) as Array<T['$inferSelect']>;
    }

    async uniqueResult(): Promise<T['$inferSelect'] | undefined> {
        return (await this.list())[0];
    }
}
```

Класс StationDao – наследник DAO – содержит методы практического задания №4, переписанные на Criteria (getName, listByIdRange), метод findAll, поиск по префиксу, метод search с динамическими фильтрами и операции get, save и delete. В методе search условие добавляется только если параметр задан (не null):

```
import { eq } from 'drizzle-orm';
import { Criteria, Order, Restrictions } from './Criteria.ts';
import { DAO } from './DAO.ts';
import { type NewStation, type Station, station } from './model.ts';

export class StationDao extends DAO {
    async createCriteria(): Promise<Criteria<typeof station>> {
        return new Criteria(await DAO.getSession(), station);
    }

    async findAll(): Promise<Station[]> {
        const criteria = await this.createCriteria();
        return criteria.addOrder(Order.asc(station.id)).list();
    }

    async getName(id: number): Promise<string | undefined> {
        const criteria = await this.createCriteria();
        const found = await criteria.add(Restrictions.eq(station.id, id)).uniqueResult();
        return found?.name;
    }

    async listByIdRange(from: number, to: number): Promise<string[]> {
        const criteria = await this.createCriteria();
        const rows = await criteria.add(Restrictions.between(station.id, from,
to)).addOrder(Order.asc(station.id)).list();
        return rows.map((row) => row.name);
    }

    async findByNameLike(prefix: string): Promise<Station[]> {
        const criteria = await this.createCriteria();
        return criteria.add(Restrictions.ilike(station.name,
`${prefix}%`)).addOrder(Order.asc(station.id)).list();
    }

    async search(name: string | null, capacityFrom: number | null, capacityTo: number | null):
Promise<Station[]> {
        const criteria = await this.createCriteria();
        if (name !== null) criteria.add(Restrictions.ilike(station.name, `%${name}%`));
        if (capacityFrom !== null) criteria.add(Restrictions.ge(station.capacity, capacityFrom));
        if (capacityTo !== null) criteria.add(Restrictions.le(station.capacity, capacityTo));
        return criteria.addOrder(Order.asc(station.id)).list();
    }

    async findByDistrict(districtId: number): Promise<Station[]> {
        const criteria = await this.createCriteria();
        return criteria.add(Restrictions.eq(station.districtId,
districtId)).addOrder(Order.asc(station.id)).list();
    }

    async get(id: number): Promise<Station | undefined> {
        const criteria = await this.createCriteria();
        return criteria.add(Restrictions.eq(station.id, id)).uniqueResult();
    }

    async save(data: NewStation): Promise<Station> {
        const session = await DAO.getSession();
        const rows = await session.insert(station).values(data).returning();
        return rows[0]!;
    }

    async delete(id: number): Promise<number> {
        const session = await DAO.getSession();
        const result = await session.delete(station).where(eq(station.id, id));
        return result.rowCount ?? 0;
```

```
    }
}
```

Класс DistrictDao работает с районами и демонстрирует связь «один ко многим»: метод getWithStations возвращает район вместе с его станциями.

```
import { eq } from 'drizzle-orm';
import { Criteria, Order, Restrictions } from './Criteria.ts';
import { DAO } from './DAO.ts';
import { district } from './model.ts';

export class DistrictDao extends DAO {
    async createCriteria(): Promise<Criteria<typeof district>> {
        return new Criteria(await DAO.getSession(), district);
    }

    async findAll(): Promise<Array<typeof district.$inferSelect>> {
        const criteria = await this.createCriteria();
        return criteria.addOrder(Order.asc(district.id)).list();
    }

    async get(id: number): Promise<typeof district.$inferSelect | undefined> {
        const criteria = await this.createCriteria();
        return criteria.add(Restrictions.eq(district.id, id)).uniqueResult();
    }

    async getWithStations(id: number) {
        const session = await DAO.getSession();
        return session.query.district.findFirst({ where: eq(district.id, id), with: { stations: true }
});
    }
}
```

5 Репозитории

Обобщённый класс JpaRepository даёт любому репозиторию готовые операции findAll, findById, save и deleteById без повторной реализации; конкретные репозитории наследуют его и указывают таблицу.

```
import { eq } from 'drizzle-orm';
import type { PgColumn, PgTable } from 'drizzle-orm/pg-core';
import { DAO } from './DAO.ts';

export class JpaRepository<T extends PgTable & { id: PgColumn }> {
    protected table: T;

    constructor(table: T) {
        this.table = table;
    }

    async findAll(): Promise<Array<T['$inferSelect']>> {
        const session = await DAO.getSession();
        return (await session.select().from(this.table as PgTable).orderBy(this.table.id)) as
Array<T['$inferSelect']>;
    }

    async findById(id: number): Promise<T['$inferSelect'] | undefined> {
        const session = await DAO.getSession();
        const rows = await session.select().from(this.table as PgTable).where(eq(this.table.id, id));
        return rows[0] as T['$inferSelect'] | undefined;
    }

    async save(data: T['$inferInsert']): Promise<T['$inferSelect']> {
        const session = await DAO.getSession();
        const rows = await session.insert(this.table).values(data).returning();
        return rows[0] as T['$inferSelect'];
    }

    async deleteById(id: number): Promise<void> {
        const session = await DAO.getSession();
        await session.delete(this.table).where(eq(this.table.id, id));
    }
}
```

Репозиторий станций добавляет метод запроса по соглашению об именах findByNameStartingWithIgnoreCase (поиск по префиксу без учёта регистра) и метод с произвольным запросом findStationsByDistrictName (соединение станций с районами и выбор по названию района – аналог @Query). Репозиторий районов не добавляет методов.

```
import { eq, ilike } from 'drizzle-orm';
import { DAO } from './DAO.ts';
import { JpaRepository } from './JpaRepository.ts';
import { district, type Station, station } from './model.ts';

export class StationRepository extends JpaRepository<typeof station> {
    constructor() {
        super(station);
    }

    async findByNameStartingWithIgnoreCase(prefix: string): Promise<Station[]> {
        const session = await DAO.getSession();
        return session.select().from(station).where(ilike(station.name,
`${prefix}%`)).orderBy(station.id);
    }

    async findStationsByDistrictName(name: string): Promise<Station[]> {
        const session = await DAO.getSession();
        const rows = await session
            .select({ station })
            .from(station)
            .innerJoin(district, eq(district.id, station.districtId))
            .where(eq(district.name, name))
            .orderBy(station.id);
        return rows.map((row) => row.station);
    }
}
```

```
import { JpaRepository } from './JpaRepository.ts';
import { district } from './model.ts';

export class DistrictRepository extends JpaRepository<typeof district> {
    constructor() {
        super(district);
    }
}
```

6 Примеры вызовов и вывод в консоль

Файл Main.ts вызывает все реализованные методы: Criteria, динамические фильтры, создание и удаление станций района в транзакции, репозитории.

```
import { DAO } from './DAO.ts';
import { DistrictDao } from './DistrictDao.ts';
import { DistrictRepository } from './DistrictRepository.ts';
import { StationDao } from './StationDao.ts';
import { StationRepository } from './StationRepository.ts';

const stationDao = new StationDao();
const districtDao = new DistrictDao();
const stationRepository = new StationRepository();
const districtRepository = new DistrictRepository();

console.log('findAll:', (await stationDao.findAll()).length);
console.log('getName(1):', await stationDao.getName(1));
console.log('listByIdRange(1, 5):', await stationDao.listByIdRange(1, 5));
console.log('findByNameLike("Пл"):', (await stationDao.findByNameLike('Пл')).map((s) => s.name));

console.log('search("Пл", null, null):', (await stationDao.search('Пл', null, null)).length);
console.log('search(null, 16, 18):', (await stationDao.search(null, 16, 18)).length);
console.log('search("Пл", 10, null):', (await stationDao.search('Пл', 10, null)).length);

const district = await districtDao.get(1);
const before = (await stationDao.findByDistrict(1)).length;
await DAO.begin();
const first = await stationDao.save({ districtId: 1, name: `Станция ПЗ5 А ${Date.now()}`, address: 'ул. Ленина,
5', capacity: 10, latitude: '55.750000', longitude: '37.600000', openedOn: '2026-11-05' });
const second = await stationDao.save({ districtId: 1, name: `Станция ПЗ5 Б ${Date.now()}`, address: 'ул. Ленина,
6', capacity: 12, latitude: '55.750000', longitude: '37.600000', openedOn: '2026-11-05' });
await DAO.commit();
console.log(`district ${district?.name}: stations ${before} -> ${(await stationDao.findByDistrict(1)).length}`);
console.log('via relation:', (await districtDao.getWithStations(1))?.stations.length);
await DAO.begin();
await stationDao.delete(first.id);
await stationDao.delete(second.id);
await DAO.commit();
console.log('after delete:', (await stationDao.findByDistrict(1)).length);

console.log('repository findAll:', (await stationRepository.findAll()).length);
console.log('repository findById(1):', (await stationRepository.findById(1))?.name);
console.log('districts:', (await districtRepository.findAll()).length);
const saved = await stationRepository.save({ districtId: 1, name: `Станция ПЗ5 В ${Date.now()}`, address: 'ул.
Ленина, 7', capacity: 8, latitude: '55.750000', longitude: '37.600000', openedOn: '2026-11-05' });
console.log('saved id:', saved.id);
await stationRepository.deleteById(saved.id);
console.log('findByNameStartingWithIgnoreCase("пл"):', (await
stationRepository.findByNameStartingWithIgnoreCase('пл')).length);
console.log(`findStationsByDistrictName("${district?.name}"):`, (await
stationRepository.findStationsByDistrictName(district?.name ?? '')).length);

await DAO.close();
```

Запуск – npm start. Вывод программы показан на рисунке 1.

```
PS ...\pz5-entities-criteria-repositories> npm start
findAll: 300
getName(1): Площадь Революции
listByIdRange(1, 5): [
  'Площадь Революции',
  'Городской сад',
  'Морской вокзал',
  'Приморский бульвар',
  'Заречье'
]
findByNameLike("Пл"): [ 'Площадь Революции' ]
search("Пл", null, null): 1
search(null, 16, 18): 29
search("Пл", 10, null): 1
district Центральный: stations 33 -> 35
via relation: 35
after delete: 33
repository findAll: 300
repository findById(1): Площадь Революции
districts: 10
saved id: 303
findByNameStartingWithIgnoreCase("пл"): 1
findStationsByDistrictName("Центральный"): 33
```

Рисунок 1 – Вывод Main

Пояснение результатов:

– findAll: 300 – Criteria без условий вернул все станции;

– getName(1) и listByIdRange(1, 5) – методы практического задания №4 на Criteria дали тот же результат, что и раньше;

– findByNameLike("Пл") – одна станция с названием, начинающимся на «Пл»;

– search – динамический фильтр: при одном заданном условии (название) найдена 1 станция; при условии только по вместимости от 16 до 18 – 29 станций; при названии и нижней границе вместимости – 1 станция; незаданные условия в запрос не попадают;

– demo связи 1 : N: в районе «Центральный» было 33 станции, после сохранения двух новых в транзакции стало 35, через связь getWithStations тоже 35, после удаления снова 33;

– репозиторий: findAll – 300 станций, findById(1) – «Площадь Революции», save вернул запись с новым идентификатором и она была удалена deleteById, findByNameStartingWithIgnoreCase("пл") нашёл станцию без учёта регистра, findStationsByDistrictName("Центральный") вернул 33 станции.

7 Выводы

1 Для всех девяти таблиц базы данных созданы сущности; связь «один ко многим» район – станция описана с обеих сторон и используется при чтении.

2 Запросы построены программным интерфейсом Criteria; методы практического задания №4 переписаны на него, добавлены findAll, поиск по префиксу и метод search, в котором условия добавляются только при наличии параметров.

3 Реализованы операции get, save и delete и демонстрация связи «один ко многим» с сохранением и удалением данных в транзакции.

4 Репозитории JpaRepository, StationRepository и DistrictRepository подключены и используются; метод по соглашению об именах и метод с соединением возвращают корректные данные.